# Intelligent Shipment Route Optimizer

*Technical Report — Aiqon AI Internship Take-Home: Optimizing Shipment Routes Under Cost, Time, and Risk Constraints*

*Mohammed Dayyan | github.com/MohammedDayyan/aiqon-shipment-optimizer*

## 1. Problem and definition of “optimal”

Choosing a shipment route is a trade-off between cost, transit time, reliability and risk, made under uncertain delay estimates. A cheaper route may be delayed more often; a faster one may cross congested or high-risk regions. Given a shipment with 3–5 candidate routes, the system must (1) evaluate every route, (2) recommend one using defined criteria and (3) explain why it beats the alternatives. **Definition.** The optimal route is the candidate with the **lowest composite penalty** over normalized cost, *expected* transit time and risk, with objective weights set by the shipment's priority profile (*time_critical*, *budget_optimized*, *risk_averse*, *balanced*). The system is **decision support**: it recommends and explains; the operator keeps the final decision.

## 2. System architecture

| 1. Priority profile<br>weights | 2. Uncertainty<br>expected time | 3. Normalizer<br>min-max [0,1] | 4. Scorer<br>weighted sum, rank | 5. Explainer<br>rank 1 vs rank 2 |
|---|---|---|---|---|

A shipment payload flows left to right through the five stages in `src/engine/`. A FastAPI service (`src/main.py`) exposes the pipeline with Pydantic v2 validation (`src/models/schemas.py`); a Tailwind dashboard (`frontend/index.html`) consumes it. Docker support is provided for the API.

```
E[T] = T_base + P_delay x D_delay
x' = (x - min) / (max - min) (per shipment, across its candidate routes)
Score = w_c*cost' + w_t*E[T]' + w_r*risk' (lower = better)
```

**Illustrative worked example** (numbers invented for explanation; weights 0.4 / 0.3 / 0.3 and risk Low=1, Medium=2, High=3 are *not* the repo's actual values). Three routes are scored; normalization is across these three candidates only.

| Route | Cost | Base days | P(delay) | Delay days | Risk | E[T] days | cost' | E[T]' | risk' | Score |
|---|---|---|---|---|---|---|---|---|---|---|
| A | 1000 | 22 | 0.45 | 6 | High | 24.7 | 0.00 | 1.00 | 1.00 | **0.60** |
| B | 1150 | 20 | 0.15 | 4 | Low | 20.6 | 0.38 | 0.50 | 0.00 | **0.30** |
| C | 1400 | 15 | 0.3 | 5 | Medium | 16.5 | 1.00 | 0.00 | 0.50 | **0.55** |

Route B wins with score 0.30. The cheapest route is A, but B costs only 15% more, is expected to arrive 17% sooner and carries lower risk, so a contrastive explanation would read: *“B costs about 15% more than A but cuts expected transit time by about 17% and carries lower risk.”* Profile weights: [insert the real weights from profiles.py]; risk mapping: [insert Low/Med/High values].

## 3. Design decisions and justification

- **Weighted scoring over full multi-objective optimization.** With 3–5 candidates the search is trivial; the real requirement is explainability. An operator can see exactly why A beat B and which weight change would flip it.
- **Expected transit time instead of raw time.** A route that is fast on paper but often delayed should not win on its advertised number; this folds reliability into one comparable value.
- **Min-max normalization.** Currency, days and ordinal risk have different scales; normalizing stops the largest unit dominating the weights.
- **Priority profiles instead of raw weights.** Operators think “this cargo is urgent”, not “w_time = 0.5”; profiles keep weights inspectable.
- **Contrastive explanation.** Operators ask “why this one and not the other?”, so the winner is compared with the runner-up.
- **Modular engine.** One module per stage, so a stage (e.g. the normalizer) can be replaced without touching the rest; the tests target each stage.

## 4. Repository structure

| Path | Purpose |
|---|---|
| `data/generator.py` | Generates synthetic shipments, 3–5 routes each. |
| `data/synthetic_shipments.json` | Generated dataset. |
| `src/main.py` | FastAPI app and endpoints. |
| `src/engine/normalizer.py, profiles.py, scorer.py, explainer.py` | Min-max scaling; profile weights; expected-time and weighted scoring; contrastive explanation. |
| `src/models/schemas.py` | Pydantic v2 request/response models. |
| `tests/test_engine.py` | Unit tests for scoring, normalization and explainability. |
| `frontend/index.html` | Single-page Tailwind dashboard. |
| `Dockerfile, docker-compose.yml, requirements.txt` | Containerization and dependencies. |

## 5. How to run (step by step)

**Prerequisites:** Python 3.11+ and pip (or Docker). Run commands from the repository root.

**Step 1 – Clone and create a virtual environment**

```
git clone https://github.com/MohammedDayyan/aiqon-shipment-optimizer.git
cd aiqon-shipment-optimizer
python -m venv .venv
```

**Step 2 – Activate it** (use the line for your shell; note the folder is **bin** on Linux/macOS but **Scripts** on Windows)

```
# Windows PowerShell : .\.venv\Scripts\Activate.ps1
# Windows cmd        : .\.venv\Scripts\activate.bat
# Git Bash (Windows) : source .venv/Scripts/activate
# Linux / macOS      : source .venv/bin/activate
```

**Step 3 – Install dependencies**

```
pip install -r requirements.txt
```

**Step 4 – Start the backend** (terminal 1). API at http://localhost:8000, Swagger UI at http://localhost:8000/docs (use "Try it out" to send a shipment and inspect the ranked routes and explanation).

```
uvicorn src.main:app --reload --port 8000
```

**Step 5 – Start the frontend** (terminal 2, same activated environment). Dashboard at http://localhost:8080.

```
python -m http.server 8080 --directory frontend
```

**Step 6 – Run the tests** (if pytest is missing: `pip install pytest`)

```
pytest tests/ -v
```

**Alternative – Docker.** `docker compose up --build` builds and starts the API service. [confirm exposed port and whether the frontend is served]

**Regenerate data (optional):** `python data/generator.py` [confirm command and seed]

**Troubleshooting.**

- *PowerShell blocks activation* ("running scripts is disabled"): run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in that window, then activate again.
- *"ModuleNotFoundError: src"*: run uvicorn from the repository root, not from inside `src/`.
- *Port already in use*: pick another (e.g. `--port 8001`) and update the API base URL used by `frontend/index.html`. [state where the URL is set]
- *Dashboard loads but shows no results*: confirm the backend is running and check the browser console; a CORS error means the API must allow the frontend origin [confirm CORS is configured in main.py].

## 6. Synthetic data

No public dataset provides route-level delay distributions, so `data/generator.py` simulates shipments with origin, destination, intermediate hubs, cost, base transit time, delay probability, delay duration and risk indicators (e.g. congestion, geopolitical). [state N shipments, seed, and how trade-offs were built in, e.g. faster = costlier, cheap = more delay-prone] Because the trade-offs were designed by the author, the data shows the framework *behaves sensibly*; it cannot show the recommendations are *correct* on real shipments.

## 7. Assumptions

- Delay probability and duration are known point estimates; delays are the only random element; one currency; cost and base time are deterministic.
- Risk categories are ordinal and mapped to numeric penalties; profile weights are hand-set judgments, not learned.
- Each shipment is optimized in isolation (no capacity limits, no interaction between shipments, no live data).
- [state whether delay/risk are modelled per leg or per route]

## 8. Evaluation

**Implemented:** unit tests for the scorer, normalizer and explainer [test count, passing], which show the code matches the design. **Not yet done:** evaluation of *decision quality*. Planned protocol: (a) compare against baselines (always cheapest / fastest / lowest risk); (b) Monte Carlo simulation of realized delays from each route's delay probability and duration, comparing realized cost, lateness and on-time rate per policy; (c) perturb profile weights (±10%, ±25%) and report how often rank 1 changes.

## 9. Limitations

- **Compensatory scoring, no hard constraints.** A very cheap route can offset very high risk; deadlines, budget caps or restricted regions cannot be enforced, so an unacceptable route can still rank first. A weighted sum also cannot reach non-convex Pareto trade-offs.

- **Per-shipment min-max is relative.** It magnifies tiny differences (a 1% cost gap becomes 0 vs 1), makes scores incomparable across shipments, and allows rank reversal when a route is added or removed.
- **Mean-only uncertainty.** E[T] hides variance and tail risk; two routes with equal E[T] can differ greatly in worst-case lateness.
- **Uncalibrated parameters.** Weights and the risk mapping are unvalidated and no sensitivity analysis is reported yet.
- **Synthetic data only; descriptive explanations.** Results reflect generator assumptions; explanations report score components but do not justify the weights.
- **Production gaps.** No authentication, persistence, monitoring or real data integration.

## 10. Planned improvements (priority order)

(1) Hard constraints and Pareto-dominance filtering before scoring. (2) Express delay in money (cost per day late) so weights have real-world meaning. (3) Monte Carlo baseline evaluation. (4) Weight-sensitivity analysis with a “close call” flag when the top two routes are within a margin. (5) Tail-aware metrics (P90 time, CVaR) and per-leg delay compounding, 1 - (1-p1)(1-p2)...(1-pn). (6) Calibrate weights on real operator decisions.